

ThreadLearn: Combining Static Race Detection with Hindsight Chain-of-Thought Fine-Tuning for Concurrency Bug Remediation

Under Review — Target Venue: ASPLOS 2027 / ICLR 2026

Lê Trí Trung Hà Văn Ân

FPT University letritrung2605@gmail.com

Abstract

Concurrency bugs in async JavaScript code often cause serious failures in production systems. Existing tools have two main weaknesses: rule-based detectors can spot suspicious patterns but cannot suggest how to fix them, while large language models (LLMs) can reason about code but tend to miss the exact line where a bug occurs and require very large models (7B–32B parameters) to work well. ThreadLearn addresses both weaknesses by combining a 5-pattern static race detector with a small language model (Qwen2.5-Coder-1.5B) that is fine-tuned using QLoRA ($r=16$, $\alpha=32$, 4-bit NF4 compression) on 783 training examples. Each training example includes a step-by-step explanation written *after* the correct fix is already known (“hindsight” reasoning). At inference time, a BM25 keyword search retrieves the 3 most relevant documents from a 2050-document knowledge base and adds them to the prompt. On a hand-crafted benchmark of 20 JavaScript concurrency bugs across 8 categories, ThreadLearn with retrieval achieves a **75% pass rate (15/20)**, compared with 40% (8/20) for the same model without fine-tuning and 30% (6/20) for GPT-3.5-turbo—a gain of +35 percentage points over the base model and +45 pp over GPT-3.5. The fine-tuning dataset (783 JSONL samples) and evaluation script (`eval_rag_merged.py`) are publicly available.

Keywords: concurrency bugs, race conditions, static analysis, LLM fine-tuning, Chain-of-Thought, JavaScript

1 Introduction

1.1 Problem Statement

Asynchronous JavaScript (Node.js) is widely used for building web backends, but the language makes it easy to write code that behaves incorrectly when multiple operations run at the same time. A large-scale study (NodeCB [1]) looked at 57 real concurrency bugs in 53 open-source Node.js projects and found that 65% involve two operations conflicting on shared data (atomicity violations), 30% involve operations completing in the wrong order, and 93% cause serious problems such as crashes, wrong database values, or the program hanging.

Despite how common these bugs are, existing tools still fall short in different ways. Rule-based detectors can find suspicious patterns but cannot explain or fix them, and they often produce false alarms on modern `async/await` code. LLM-based tools can reason about whole functions but cannot pinpoint which exact line caused the bug and need very large models to achieve good results.

1.2 Gap Analysis

We identify four concrete gaps in current tools:

- G1** Rule-based detectors can only find bugs; they cannot fix them.
- G2** PCWMs [2] reaches 75.2% accuracy but ignores the structural information from static analysis and needs 27K training samples with a large base model.
- G3** No existing tool uses static analysis output as additional context to guide LLM reasoning.
- G4** No end-to-end tool covers the full workflow of detect → explain → fix for JavaScript (Node.js).

1.3 Key Insight

The output of a static detector—which pattern was matched, on which lines, and why it is dangerous—gives the LLM concrete, line-level evidence to reason from, instead of guessing from general code patterns. A small model (1.5B parameters) trained with this kind of grounded, step-by-step reasoning outperforms a much larger model given only a generic prompt, because it has learned to follow structured evidence rather than surface-level text similarity.

1.4 Contributions

This paper makes four contributions:

1. **race_detector**: a 5-pattern static analyzer for JavaScript (Node.js) that works together with an AST code preprocessor (§3.2).
2. **Hindsight CoT dataset**: a training dataset of (`code`, `detector_output`, `reasoning_trace`, `fix`) examples, where the reasoning is written with knowledge of the correct fix (§3.3).
3. **ThreadLearn pipeline**: a complete system that detects, formats, retrieves context, and generates a fix, served via a FastAPI web endpoint (§3.4).
4. **Evaluation**: comparison against NodeCB and PCWMs across fix pass rate, false-positive rate, and response latency (§5).

Figure 1 — System Overview

code input → AST preprocessor → race detector → report formatter →
 RAG pipeline → fine-tuned LLM → fix output
Baselines: NodeCB rule-only, PCWMs LLM-only (shown for comparison)

Figure 1: ThreadLearn end-to-end system overview.

2 Background and Motivation

2.1 Concurrency Bug Taxonomy

Table 1 lists the types of concurrency bugs found in the NodeCB study and shows how each type appears in Node.js code.

Table 1: Concurrency bug types (from NodeCB study) with Node.js examples.

Bug Type	Rate	Example
Atomicity violation	65%	Two database calls overlap, causing a lost update
Order violation	30%	Upload finishes before the file-create callback runs
Starvation	5%	<code>setImmediate</code> never runs because the I/O queue is full
<code>var i + setTimeout</code>	JS	Loop variable captured by reference; all callbacks see the final value

2.2 Limitations of Prior Approaches

Table 2: Weaknesses of prior tools and key observations from this work.

Approach	Weakness / Observation
NodeCB study	Uses regex patterns; high false-positive rate; no fix generation
PCWMs	75.2% accuracy; ignores static structure; requires a large model (7B–32B parameters)
O1	Detector output (line range, pattern name) gives the LLM a concrete reason for each bug instead of a guess
O2	A small model (1.5B) trained with grounded step-by-step reasoning outperforms a large model given only a generic prompt

2.3 AST Preprocessing Foundation

ThreadLearn’s AST preprocessor provides four helper functions used by both the race detector and the document retrieval stage. AST (Abstract Syntax Tree) parsing converts raw source code into a structured tree of named nodes—functions, variables, operators—so downstream tools can work with precise code structure instead of plain text.

3 Design

3.1 System Architecture Overview

ThreadLearn has five independent modules (Table 4). Each module has a single clear job; they communicate through simple data structures so any one module can be replaced

Table 3: AST preprocessor functions and what they do.

Function	Purpose	Notes
<code>stripComments</code>	Remove comments from code	esprima range-delete for JS; <code>ast.unparse</code> for Python
<code>normalizeWhitespace</code>	Remove extra blank lines/spaces	esprima token stream for JS; <code>ast.unparse</code> for Python
<code>extractFunctions</code>	List all functions in a file	esprima AST walk: finds arrow functions and anonymous expressions too
<code>extract_keywords</code>	Pick top-20 identifier names	esprima <code>Identifier</code> tokens; used to build the BM25 search query

Figure 2 — Component Architecture

AST preprocessor → race detector → report formatter → RAG pipeline → fine-tuned LLM

Figure 2: ThreadLearn five-component architecture with data flow.

without changing the others.

Table 4: ThreadLearn modules and their roles.

Module	Job
<code>ast_preprocessor</code>	Clean and normalize the input code
<code>race_detector</code>	Find 5 known JS race condition patterns
<code>report_formatter</code>	Turn a pattern name into a severity level and fix hint
<code>rag_pipeline</code>	Search for related examples; add them to the LLM prompt
Fine-tuned LLM	Generate the corrected code with step-by-step reasoning
FastAPI server	Expose the <code>/analyze</code> HTTP endpoint

3.2 Static Race Detector

The detector reads a source file line by line in $O(n)$ time and outputs a structured report: `{pattern_id, line_range, description}`. Table 5 lists all 5 patterns it checks for. TypeScript files are handled the same way as JavaScript.

Example: closure loop variable (JavaScript).

Listing 1: Bug: `var i` is shared across all callbacks.

```

1 for (var i = 0; i < 5; i++) {
2   setTimeout(function() {
3     console.log(i); // always prints 5
4   }, 100);
5 }
```

Listing 2: Fix: `let i` creates a separate copy for each loop iteration.

Table 5: The 5 race condition patterns detected by ThreadLearn (JavaScript only).

Group	Pattern ID	Lang
Closure/async	<code>closure_loop_var</code>	JS
	<code>shared_var_settimeout</code>	JS
	<code>promise_no_await</code>	JS
Shared state	<code>concurrent_write_array</code>	JS
	<code>counter_no_atomic</code>	JS

```

1 for (let i = 0; i < 5; i++) {
2   setTimeout(function() {
3     console.log(i); // prints 0, 1, 2, 3, 4
4   }, 100);
5 }

```

3.3 Hindsight Chain-of-Thought Fine-Tuning

Inspired by PCWMs [2], we build training examples of the form:

(*code*, *detector_output*, *reasoning_trace*, *fix*)

where *detector_output* is the structured report from the race detector, and *reasoning_trace* is a step-by-step explanation written by a teacher model *after* the correct fix is already known. Writing the reasoning in hindsight—starting from the answer and explaining why it is right—produces cleaner, more consistent training signal than asking a model to reason forward under uncertainty.

Table 6: Fine-tuning settings.

Setting	Value
Base model	Qwen2.5-Coder-1.5B
Method	QLoRA: $r=16$, $\alpha=32$, 4-bit NF4 weight compression
Framework	SFTTrainer (trl $\geq 1.5.0$)
Grounding input	<code>pattern_id</code> + <code>line_range</code> from the detector
<i>Rejected alternatives</i>	
Full fine-tune	Too expensive to run on available hardware
Retrieval-only	Finds relevant documents but cannot generate a fix

3.4 RAG-Augmented Fix Pipeline

1. **Extract keywords:** `ast_preprocessor.extract_keywords()` parses the input JavaScript using `esprima`, walks the syntax tree, and collects all `Identifier` tokens—function names, API calls, variable names—while skipping JS reserved words (`if`, `const`, `async`, ...). The result is a list of up to 20 meaningful names used as a search query. Because the names are taken directly from the syntax tree, CamelCase names like `appendFile` and `setTimeout` stay intact instead of being split into separate words.

2. **Retrieve documents:** BM25 scores the query against 2,050 knowledge-base documents and returns the top-3 most relevant ones. Keeping API names intact means the query words match the document words exactly, giving better search results than a plain text tokenizer.
3. **Format:** `report_formatter` converts the `pattern_id` into a human-readable severity label and a short fix hint.
4. **Generate:** the fine-tuned LLM receives the 3 retrieved documents prepended to the prompt (using the same plain completion format as during training) and outputs the corrected code.

Why AST keyword extraction beats regex for search. A regex tokenizer splits `setTimeout` into `["set", "timeout"]` and `appendFile` into `["append", "file"]`, creating a noisy query where each word is too generic to find the right documents. `esprima` reads the actual syntax tree and returns identifier tokens as-is, so the search query contains the exact API names that appear in the knowledge base—leading to more precise retrieval for JavaScript concurrency patterns.

3.5 Rejected Design Alternatives

Table 7: Design choices that were tried and rejected.

Alternative	Why Rejected
LLM-only detection	Too many false positives; cannot point to the exact buggy line
Static analysis only	Finds the bug but cannot write a fix
Large model (7B+)	Does not fit in GPU memory on a laptop GPU
Vector search (FAISS)	Requires a separate embedding model (500 MB+ download); exact API-name matching gives better results for code search than semantic similarity

4 Implementation

Table 8: Implementation summary.

Item	Detail
Language / framework	Python 3.10+, FastAPI
Total lines of code	≈1,200
<code>race_detector.py</code>	320 lines, 10 detectors, linear scan
Model adapter files	LoRA SafeTensors at <code>./threadlearn-qwen2.5-coder-1.5b/final_adapter</code>
Infrastructure	Docker: Redis cache (localhost:6379), Atlas MongoDB
Concurrency control	One request at a time to the LLM (semaphore in FastAPI)
Fallback behavior	System keeps running if <code>ast_preprocessor</code> cannot be imported (useful for CI/C)

5 Evaluation

5.1 Experimental Setup

Table 9: Evaluation setup.

Item	Detail
Baseline 1	Qwen2.5-Coder-1.5B without fine-tuning, same prompt format
Baseline 2	GPT-3.5-turbo with no examples, temperature 0, OpenAI API
Our system	ThreadLearn: fine-tuned model + BM25 document retrieval
Test suite	20 hand-written JS concurrency bugs across 8 categories
Prompt format	"Convert to concurrent JavaScript:\n\n{code}\n" (matches training)
Scoring	PASS = output contains the correct fix pattern; PARTIAL = code generated but fix pattern not present
Hardware	NVIDIA RTX 4060 Laptop GPU, float16 precision
Date	2026-06-11

5.2 End-to-End Fix Quality Comparison

Table 10 shows pass rates on the 20-case benchmark. A test is scored PASS if the generated code contains the correct fix (e.g. `Promise.all()`, `appendFile`, `try {}`, arrow function), PARTIAL if code was generated but the fix pattern was not present, and FAIL if no code was generated at all.

Table 10: Fix quality on 20-case JS concurrency benchmark (2026-06-11).

Method	Pass	Partial	Fail	Pass Rate
GPT-3.5-turbo (no examples)	6	11	3	30%
Qwen2.5-Coder-1.5B (no fine-tuning)	8	9	3	40%
ThreadLearn RAW (fine-tuned, no retrieval)	14	6	0	70%
ThreadLearn + RAG (ours)	15	5	0	75%

Three findings stand out. (1) Fine-tuning alone gives +30 pp over the base model (70% vs. 40%), and this gain depends entirely on using the correct prompt format ("Convert to concurrent JavaScript:\n\n{code}\n")—an earlier run with the wrong chat-template prompt scored only 35%. (2) Adding BM25 document retrieval gives another +5 pp (75% vs. 70%) by providing relevant context for harder cases (e.g. the upsert pattern needed for test_03). (3) ThreadLearn produces *zero* FAIL cases in both modes—it always generates some code—while GPT-3.5 and the base model each fail to produce any code on 3 tests.

Figure 3 — Pass Rate Bar Chart

4 configurations: GPT-3.5 (30%), Qwen base (40%), ThreadLearn RAW (70%), ThreadLearn+RAG (75%)
(replace with pgfplots bar chart)

Figure 3: Pass rate comparison on 20-case JS concurrency benchmark.

5.3 Per-Category Fix Quality

Table 11 breaks down ThreadLearn+RAG results by bug category.

Table 11: ThreadLearn+RAG pass rate by bug category (20-case benchmark).

Category	Expected Fix	Pass/Total	Rate
Race Condition	async/await, appendFile, upsert	3/5	60%
Event Loop Blocking	yield generator, fs.promises, renderAsync	5/5	100%
Unhandled Rejection	try/catch + next(error)	1/1	100%
Double Callback	return before second callback	0/1	0%
Zalgo	process.nextTick to make async consistent	0/1	0%
Context Loss	arrow function to keep this	1/1	100%
Callback Hell	rewrite with async/await	1/1	100%
Resource Exhaustion	limit concurrency with slice	1/1	100%
Sequential Awaits	run in parallel with Promise.all	1/1	100%
Missing Promise.all	add Promise.all	1/1	100%
Buffer Leak	add stream error handler	0/1	0%
Event Loop Ordering	Promise.resolve().then	1/1	100%
Total		15/20	75%

ThreadLearn reaches 100% on 8 of 12 categories. The three failing categories—Zalgo (`process.nextTick` normalization), Double Callback (return-guard pattern), and Buffer Leak (stream error handler)—are rare and unusual patterns that the 1.5B model has not seen enough examples of during training to generalize reliably.

Figure 4 — Per-Category Pass Rate

12 categories, ThreadLearn+RAG vs. base model
(replace with pgfplots horizontal bar chart)

Figure 4: Per-category pass rate: ThreadLearn+RAG vs. Qwen2.5 base.

5.4 Ablation Study

Table 12: How much each component contributes (20-case benchmark).

Configuration	Pass/20	Pass Rate
GPT-3.5-turbo (no fine-tune, no retrieval)	6	30%
Qwen2.5-Coder-1.5B base (no fine-tune, no retrieval)	8	40%
ThreadLearn RAW (fine-tuned only, no retrieval)	14	70%
ThreadLearn + RAG (fine-tuned + retrieved context)	15	75%
<i>Gain from fine-tuning</i>	+30 pp over base model	
<i>Gain from RAG</i>	+5 pp over fine-tune only	
<i>Wrong prompt format</i>	35% (chat template) vs. 70% (training format)	

The biggest single factor is the prompt format. Using the chat template (`apply_chat_template`) instead of the plain completion format that the model was trained on drops the pass rate from 70% to 35%—a −35 pp drop caused entirely by a format mismatch, which hid most of the benefit from fine-tuning during an early evaluation run.

5.5 Scalability and Latency

Table 13: Server throughput and response time (mock LLM, 10 parallel clients, N=200 requests).

Metric	Value
Throughput	86 requests/second
Median latency	3.2 ms
95th percentile	112 ms
99th percentile	129 ms
Why P95 is high	When 10 clients send requests at the same time, they queue behind the single-LLM semaphore; the 95th-percentile wait jumps from 8 ms (one client) to 112 ms (10 clients)

6 Related Work

Table 14: How ThreadLearn differs from prior tools.

Group	Examples	Key Difference
Static race detectors	ThreadSanitizer, Helgrind, ESLint async rules	Find bugs but cannot fix them; no LLM reasoning
Empirical bug studies	NodeCB [1], Android concurrency bugs	Provide the bug taxonomy we build on; no automated fix
LLM for code analysis	PCWMs [2], CodeBERT, CodeT5	LLM-only; ignores static structure; needs large models
Fine-tuning for code	QLoRA [3], SFT on code, PEFT	Same training technique; not targeting concurrency bugs

Table 15: Which tools can detect bugs, generate fixes, handle JavaScript, and use fine-tuning.

Tool	Detect	Fix	JS	Fine-tuned
ThreadSanitizer	✓	×	×	×
ESLint async	✓	×	✓	×
NodeCB (rules)	✓	×	✓	×
PCWMs (LLM-only)	✓	✓	✓	✓
ThreadLearn	✓	✓	✓	✓

Feature comparison.

7 Conclusion

Concurrency bugs remain a leading cause of failures in async JavaScript applications, yet no existing tool both finds them and fixes them reliably using a small model. ThreadLearn combines a 5-pattern static race detector with a small language model (Qwen2.5-Coder-1.5B) fine-tuned on step-by-step reasoning examples and a BM25 document retrieval pipeline to deliver an end-to-end bug-fix system for JavaScript (Node.js). On a 20-case benchmark covering 8 bug categories, ThreadLearn with retrieval achieves a **75% pass rate (15/20)**, compared with 40% for the same model without fine-tuning and 30% for GPT-3.5-turbo—a +35 pp improvement showing that the right training format and domain-specific fine-tuning significantly improve fix quality even at 1.5B parameters.

Future work. Expand to TypeScript AST (replace the current regex fallback); add MPI/CUDA patterns (following PCWMs direction).

References

- [1] T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Wang, and K. V. Vishwanath. NodeCB: Understanding Concurrency Bugs in Node.js. In *Proc. 32nd IEEE/ACM ASE*, pp. 87–98, 2017.
- [2] G. Singh, A. Guha, B. Kailkhura, and H. Menon. PCWMs: Prompting with Chain-of-Thought World Models for Concurrency Bug Detection. *arXiv:2604.20926v2*, 2026.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *Proc. NeurIPS*, 2023.
- [4] K. Serebryany and T. Iskhodzhanov. ThreadSanitizer: Data Race Detection in Practice. In *Proc. WBIA*, pp. 62–71, 2009.
- [5] Qwen Team. Qwen2.5-Coder Technical Report. *arXiv:2409.12186*, 2024.